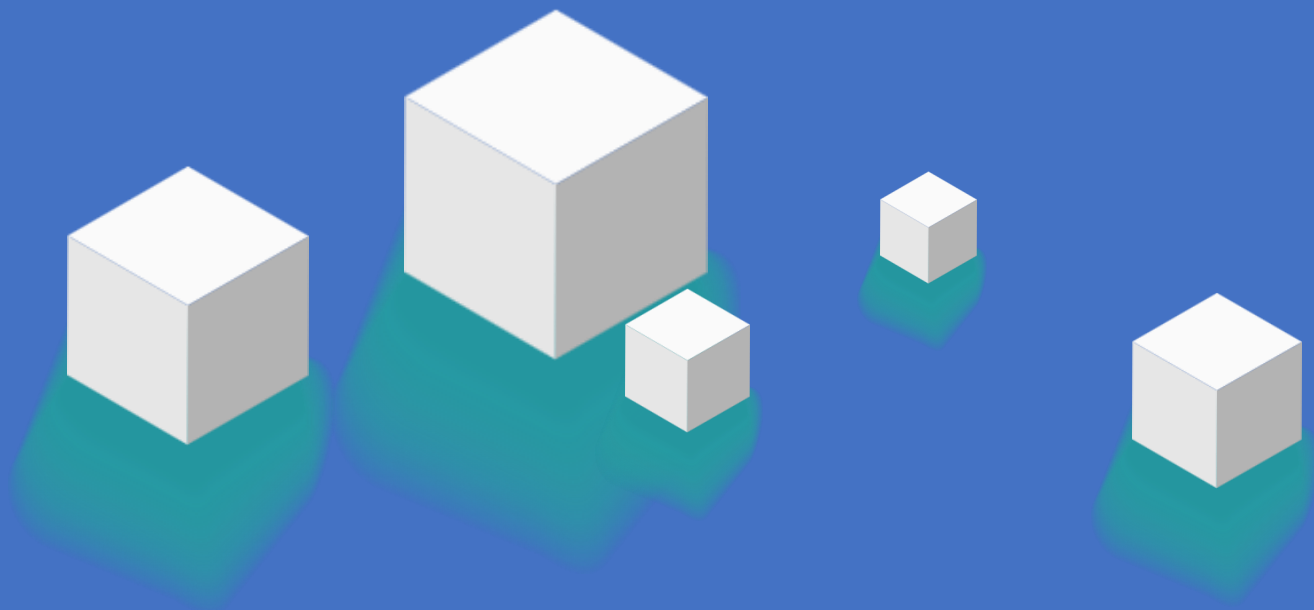


RAG技术与应用



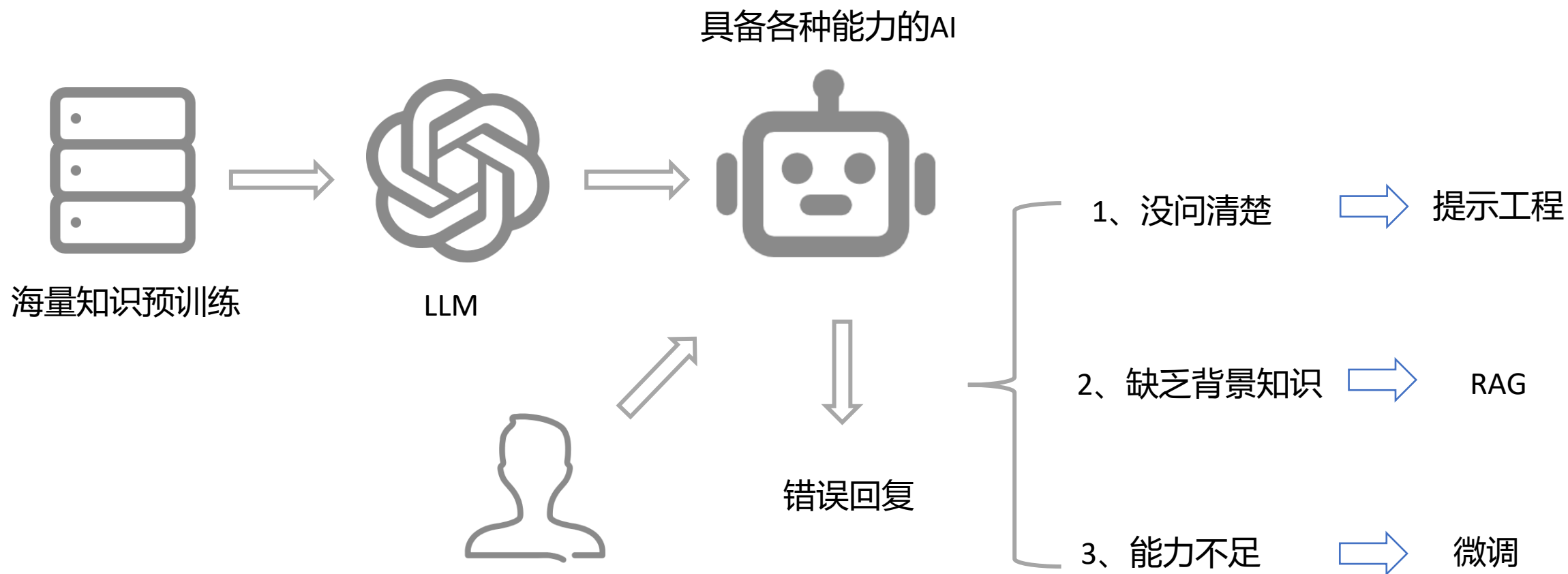
>> 今天的学习目标

RAG技术与应用

- 大模型应用开发的三种模式
- RAG的核心原理与流程
- NativeRAG
- Embedding模型选择
- CASE：DeepSeek + Faiss搭建本地知识库检索
- Query改写
- Query联网搜索

大模型应用开发

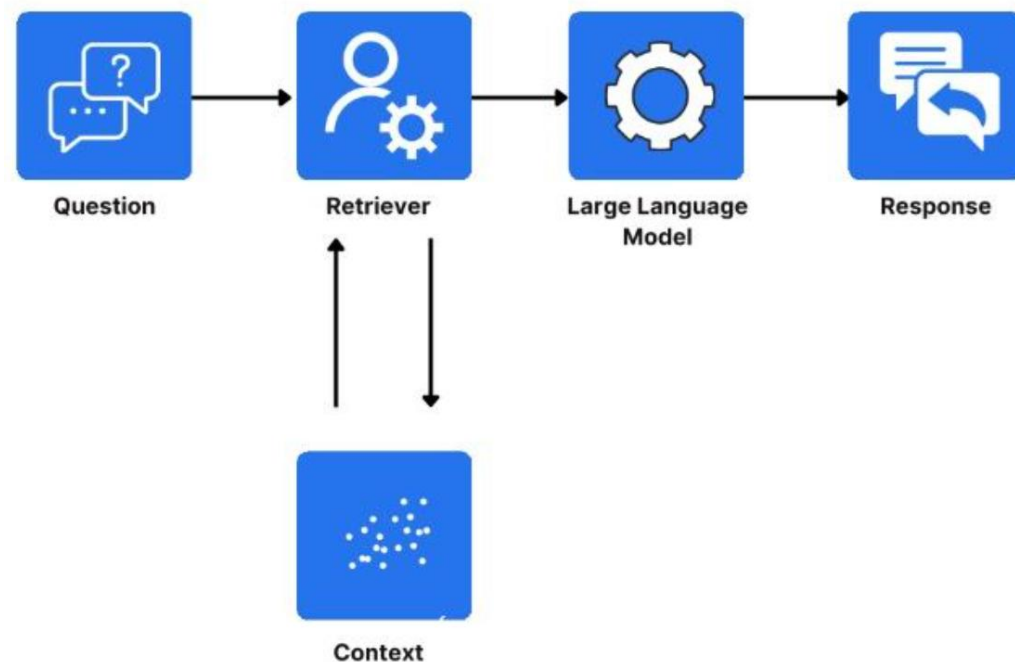
Thinking: 提示工程 VS RAG VS 微调, 什么时候使用?



什么是RAG

RAG（Retrieval-Augmented Generation）：

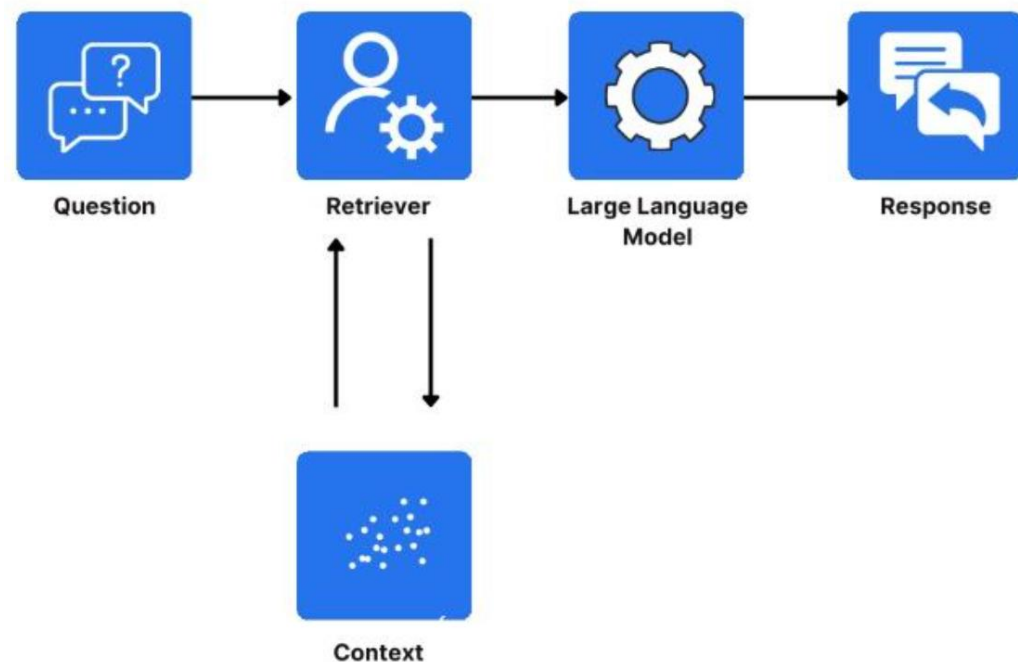
- 检索增强生成，是一种结合信息检索（Retrieval）和文本生成（Generation）的技术
- RAG技术通过实时检索相关文档或信息，并将其作为上下文输入到生成模型中，从而提高生成结果的时效性和准确性。



RAG的优势

Thinking: RAG的优势是什么？

- **解决知识时效性问题：**大模型的训练数据通常是静态的，无法涵盖最新信息，而RAG可以检索外部知识库实时更新信息。
- **减少模型幻觉：**通过引入外部知识，RAG能够减少模型生成虚假或不准确内容的可能性。
- **提升专业领域回答质量：**RAG能够结合垂直领域的专业知识库，生成更具专业深度的回答



RAG的核心原理与流程

Step1，数据预处理

- **知识库构建**：收集并整理文档、网页、数据库等多源数据，构建外部知识库。
- **文档分块**：将文档切分为适当大小的片段（**chunks**），以便后续检索。分块策略需要在语义完整性与检索效率之间取得平衡。
- **向量化处理**：使用嵌入模型（如BGE、M3E、Chinese-Alpaca-2等）将文本块转换为向量，并存储在向量数据库中

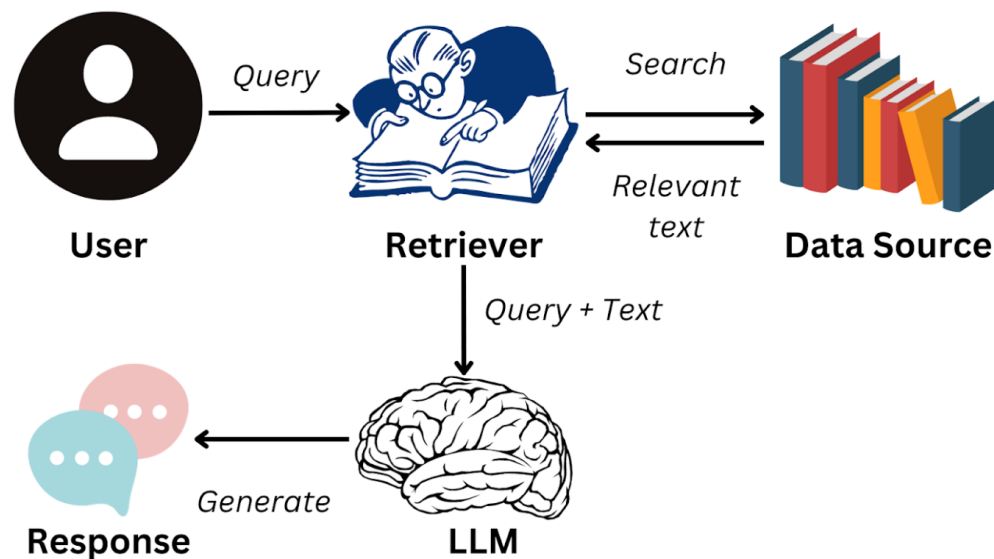
RAG的核心原理与流程

Step2, 检索阶段

- **查询处理**：将用户输入的问题转换为向量，并在向量数据库中进行相似度检索，找到最相关的文本片段。
- **重排序**：对检索结果进行相关性排序，选择最相关的片段作为生成阶段的输入

Step3, 生成阶段

- **上下文组装**：将检索到的文本片段与用户问题结合，形成增强的上下文输入。
- **生成回答**：大语言模型基于增强的上下文生成最终回答。



NativeRAG

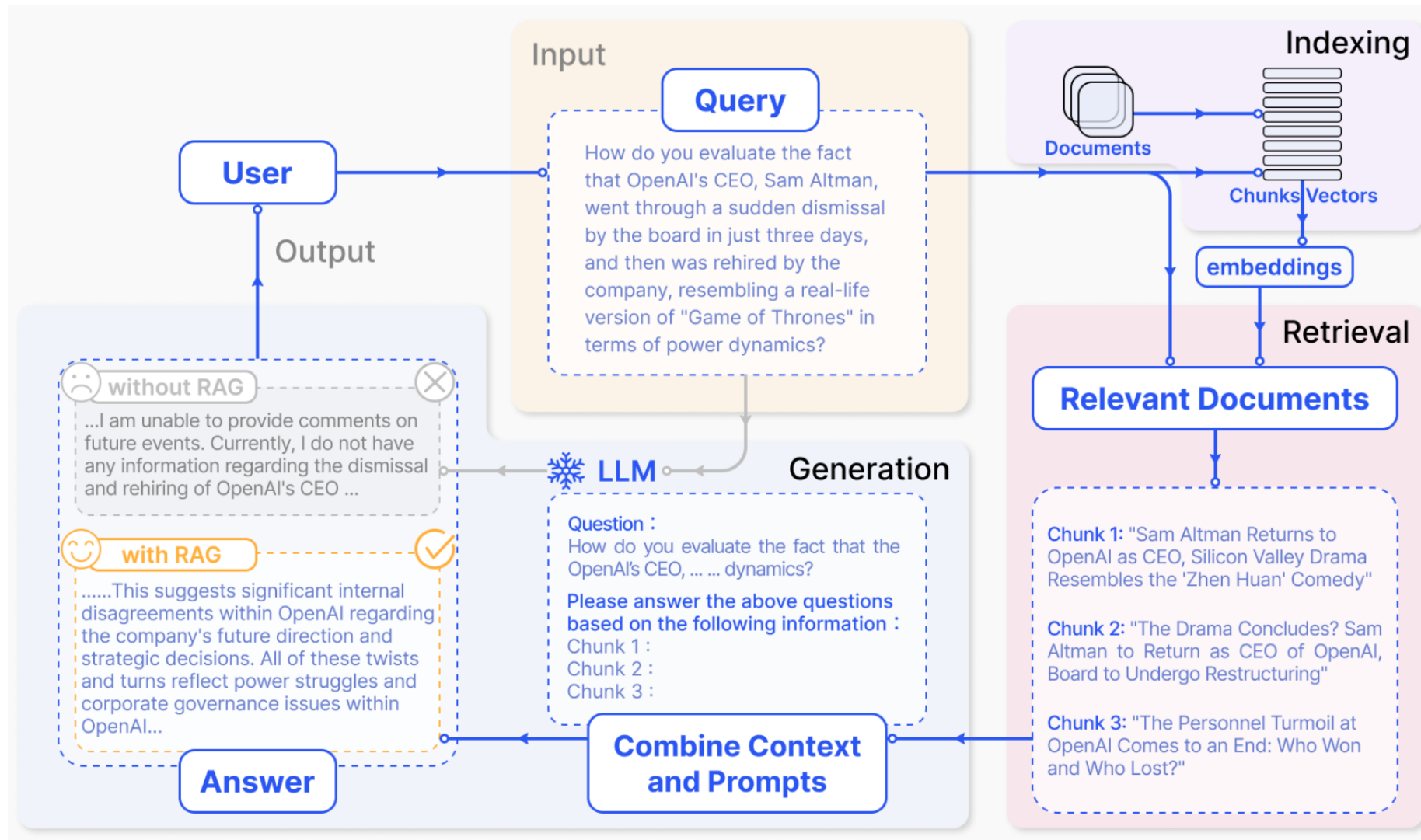
RAG的步骤:

Indexing => 如何更好地把知识存起来。

Retrieval => 如何在大量的知识中，找到一小部分有用的，给到模型参考。

Generation => 如何结合用户的提问和检索到的知识，让模型生成有用的答案。

这三个步骤虽然看似简单，但在 RAG 应用从构建到落地实施的整个过程中，涉及较多复杂的工作内容



Embedding模型选择

Embedding模型选择

Summary Performance per task Task information

Rank (Borda)	Model	Zero-shot	Clustering	Instruction Retrieval	Multilabel Classification	Pair Classification	Reranking	Retrieval	STS
1	Linq-Embed-Mistral	99%	0.27	0.94	24.77	80.43	64.37	58.69	74.86
2	gte-Qwen2-7B-instruct	⚠ NA	0.36	4.94	25.48	85.13	65.55	60.08	73.98
3	multilingual-e5-large-instruct	99%	0.54	-0.40	22.91	80.86	62.61	57.12	76.81
4	SFR-Embedding-Mistral	96%	0.57	0.16	24.55	80.29	64.19	59.44	74.79
5	GritLM-7B	99%	0.48	3.45	22.77	79.94	63.78	58.31	73.33
6	GritLM-8x7B	99%	0.88	2.44	24.43	79.73	62.61	57.54	73.16
7	e5-mistral-7b-instruct	99%	0.39	-0.62	22.20	81.12	63.82	55.75	74.02
8	Cohere-embed-multilingual-v3.0	⚠ NA	0.61	-1.89	22.74	79.88	64.07	59.16	74.80
9	gte-Qwen2-1.5B-instruct	⚠ NA	0.59	0.74	24.02	81.58	62.58	60.78	71.61
10	bilingual-embedding-large	98%	0.24	-3.04	22.36	79.83	61.42	55.10	77.81
11	text-embedding-3-large	⚠ NA	0.49	-2.68	22.03	79.17	63.89	59.27	71.68

<https://huggingface.co/spaces/mteb/leaderboard> 比较了1000多种语言中的100多种文本嵌入模型

完整榜单下载见 Embedding榜单-202503.csv

Embedding模型选择

Thinking: 有哪些常见的Embedding模型？

1、通用文本嵌入模型

BGE-M3（智源研究院）

- 特点：支持100+语言，输入长度达8192 tokens，融合密集、稀疏、多向量混合检索，适合跨语言长文档检索。
- 适用场景：跨语言长文档检索、高精度RAG应用。
- 文件大小：2.3G

text-embedding-3-large（OpenAI）

- 特点：向量维度3072，长文本语义捕捉能力强，英文表现优秀。
- 适用场景：英文内容优先的全球化应用。

Jina-embeddings-v2-small（Jina AI）

- 特点：参数量仅35M，支持实时推理（RT<50ms），适合轻量化部署。
- 适用场景：轻量级文本处理、实时推理任务。

2、中文嵌入模型

xiaobu-embedding-v2

- 特点：针对中文语义优化，语义理解能力强。
- 适用场景：中文文本分类、语义检索。

M3E-Base

- 特点：针对中文优化的轻量模型，适合本地私有化部署。
- 适用场景：中文法律、医疗领域检索任务。
- 文件大小：0.4G（m3e-base）

stella-mrl-large-zh-v3.5-1792

- 特点：处理大规模中文数据能力强，捕捉细微语义关系。
- 适用场景：中文文本高级语义分析、自然语言处理任务。

Embedding模型选择

3、指令驱动与复杂任务模型

gte-Qwen2-7B-instruct（阿里巴巴）

- 特点：基于Qwen大模型微调，支持代码与文本跨模态检索。
- 适用场景：复杂指令驱动任务、智能问答系统。

E5-mistral-7B（Microsoft）

- 特点：基于Mistral架构，Zero-shot任务表现优异。
- 适用场景：动态调整语义密度的复杂系统。

4、企业级与复杂系统

BGE-M3（智源研究院）

- 特点：适合企业级部署，支持混合检索。
- 适用场景：企业级语义检索、复杂RAG应用。

E5-mistral-7B（Microsoft）

- 特点：适合企业级部署，支持指令微调。
- 适用场景：需要动态调整语义密度的复杂系统。

CASE: bge-m3 使用

```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3',

                        use_fp16=True) # Setting use_fp16 to True speeds up
computation with a slight performance degradation

sentences_1 = ["What is BGE M3?", "Defination of BM25"]

sentences_2 = ["BGE M3 is an embedding model supporting dense
retrieval, lexical matching and multi-vector interaction.",

               "BM25 is a bag-of-words retrieval function that ranks a set of
documents based on the query terms appearing in each document"]
```

```
embeddings_1 = model.encode(sentences_1,

                             batch_size=12,

                             max_length=8192, # If you don't need such a
long length, you can set a smaller value to speed up the
encoding process.

                             )['dense_vecs']

embeddings_2 = model.encode(sentences_2)['dense_vecs']

similarity = embeddings_1 @ embeddings_2.T

print(similarity)

[[0.626  0.3477]

 [0.3499 0.678 ]]
```

CASE: bge-m3 使用

`similarity = embeddings_1 @ embeddings_2.T`

在计算两组嵌入向量（`embeddings`）之间的相似度矩阵。

`embeddings_1` 包含了第一组句子 (`sentences_1`) 的嵌入向量，
形状为 [`sentences_1`的数量, 嵌入维度]

`embeddings_2` 包含了第二组句子 (`sentences_2`) 的嵌入向量，
形状为 [`sentences_2`的数量, 嵌入维度]

`embeddings_2.T` 是对 `embeddings_2` 进行转置操作，形状变为
[嵌入维度, `sentences_2`的数量]

`@` 符号在Python中表示矩阵乘法运算

=> 通过矩阵乘法计算了两组句子之间的余弦相似度矩阵。结果 `similarity` 的形状是 [`sentences_1`的数量, `sentences_2`的数量]。

[[0.626 0.3477]

[0.3499 0.678]]

可以看出：

"What is BGE M3?" 与 "BGE M3 is an embedding model..." 的相似度为 0.6265（较高）

"What is BGE M3?" 与 "BM25 is a bag-of-words retrieval function..." 的相似度为 0.3477（较低）

"Defination of BM25" 与 "BGE M3 is an embedding model..." 的相似度为 0.3499（较低）

"Defination of BM25" 与 "BM25 is a bag-of-words retrieval function..." 的相似度为 0.678（较高）

CASE: gte-qwen2 使用

```
from sentence_transformers import SentenceTransformer
model_dir = "/root/autodl-tmp/models/iic/gte_Qwen2-1___5B-instruct"
model = SentenceTransformer(model_dir, trust_remote_code=True)
# In case you want to reduce the maximum length:
model.max_seq_length = 8192
queries = [
    "how much protein should a female eat",
    "summit define",
]
documents = [
    "As a general guideline, the CDC's average requirement of protein for women
ages 19 to 70 is 46 grams per day. But, as you can see from this chart, you'll need
to increase that if you're expecting or training for a marathon. Check out the
chart below to see how much protein you should be eating each day.",
```

```
"Definition of summit for English Language Learners. : 1 the highest
point of a mountain : the top of a mountain. : 2 the highest level. : 3 a
meeting or series of meetings between the leaders of two or more
governments.",
]
```

```
query_embeddings = model.encode(queries, prompt_name="query")
document_embeddings = model.encode(documents)
```

```
scores = (query_embeddings @ document_embeddings.T) * 100
print(scores.tolist())
```

```
[[78.49691772460938, 17.04286003112793],
 [14.924489974975586, 75.37960815429688]]
```

CASE: gte-qwen2 使用

```
import torch
import torch.nn.functional as F
from torch import Tensor
from modelscope import AutoTokenizer, AutoModel

# 定义最后一个token池化函数
# 该函数从最后的隐藏状态中提取每个序列的最后一个有效token的表示
def last_token_pool(last_hidden_states: Tensor,
                    attention_mask: Tensor) -> Tensor:
    left_padding = (attention_mask[:, -1].sum() == attention_mask.shape[0])
    if left_padding:
        return last_hidden_states[:, -1]
    else:
        sequence_lengths = attention_mask.sum(dim=1) - 1
        batch_size = last_hidden_states.shape[0]
        return last_hidden_states[torch.arange(batch_size,
                                                device=last_hidden_states.device),
                                sequence_lengths]
```

```
# 将任务描述和查询组合成特定格式的指令
def get_detailed_instruct(task_description: str, query: str) -> str:
    return f'Instruct: {task_description}\nQuery: {query}'

task = 'Given a web search query, retrieve relevant passages that answer the query'

queries = [
    get_detailed_instruct(task, 'how much protein should a female eat'), # 女性应该摄入多少蛋白质
    get_detailed_instruct(task, 'summit define') # summit（顶峰）的定义
]

# 检索文档
documents = [
    "As a general guideline, the CDC's average requirement of protein for women ages 19 to 70 is 46 grams per day. But, as you can see from this chart, you'll need to increase that if you're expecting or training for a marathon. Check out the chart below to see how much protein you should be eating"
```


CASE: gte-qwen2 使用

```
each day.", # 关于女性蛋白质摄入量的文档
    "Definition of summit for English Language Learners. : 1 the highest point
of a mountain : the top of a mountain. : 2 the highest level. : 3 a meeting or
series of meetings between the leaders of two or more governments." # 关
于summit定义的文档
]
# 将查询和文档合并为一个输入文本列表
input_texts = queries + documents

# 设置模型路径
model_dir = "/root/autodl-tmp/models/iic/gte_Qwen2-1___5B-instruct"
# 加载分词器, trust_remote_code=True允许使用远程代码
tokenizer = AutoTokenizer.from_pretrained(model_dir,
trust_remote_code=True)
# 加载模型
model = AutoModel.from_pretrained(model_dir, trust_remote_code=True)
```

```
max_length = 8192
batch_dict = tokenizer(input_texts, max_length=max_length, padding=True,
truncation=True, return_tensors='pt')
outputs = model(**batch_dict)
# 使用last_token_pool函数从最后的隐藏状态中提取每个序列的表示
embeddings = last_token_pool(outputs.last_hidden_state,
batch_dict['attention_mask'])

embeddings = F.normalize(embeddings, p=2, dim=1)
scores = (embeddings[:2] @ embeddings[2:].T) * 100
print(scores.tolist())
```

```
[[78.49689483642578, 17.042858123779297],
 [14.924483299255371, 75.37962341308594]]
```

Summary

gte-Qwen2-7B-instruct 是基于 Qwen2 的指令优化型嵌入模型

指令优化： 经过大量指令-响应对的训练，特别擅长理解和生成高质量的文本。

性能表现： 在文本生成、问答系统、文本分类、情感分析、命名实体识别和语义匹配等任务中表现优异。

适合场景： 适合复杂问答系统，处理复杂的多步推理问题，能够生成准确且自然的答案。

优势：

- 指令理解和执行能力强，适合复杂的指令驱动任务。
- 多语言支持，能够处理多种语言的文本。
- 在文本生成和语义理解任务中表现优异。

局限：

- 计算资源需求较高，适合资源充足的环境。

CASE: DeepSeek + Faiss 搭建本地知识库检索

CASE: DeepSeek + Faiss搭建本地知识库检索

百度文库 - 好好学习，天天向上

上海浦东发展银行西安分行 个金客户经理管理考核暂行办法

第一章 总 则

第一条 为保证我分行个金客户经理制的顺利实施，有效调动个金客户经理的积极性，促进个金业务快速、稳定地发展，根据总行《上海浦东发展银行个人金融营销体系建设方案（试行）》要求，特制定《上海浦东发展银行西安分行个金客户经理管理考核暂行办法（试行）》（以下简称本办法）。

第二条 个金客户经理系指各支行（营业部）从事个人金融产品营销与市场开拓，为我行个人客户提供综合银行服务的我行市场人员。

第三条 考核内容分为两大类，即个人业绩考核、工作质量考核。个人业绩包括个人资产业务、负债业务、卡业务。工作质量指个人业务的资产质量。

第四条 为规范激励规则，客户经理的技术职务和薪资实行每年考核浮动。客户经理的奖金实行每季度考核浮动，即客户经理按其考核内容得分与行员等级结合，享受对应的行员等级待遇。



客户经理被投诉了，投诉一次扣多少分

根据文件内容，客户经理被投诉一次扣2分。具体规定如下：

（二）服务质量考核：

2. 客户服务效率低，态度生硬或不及时为客户提供维护服务，有客户投诉的，每投诉一次扣2分。

客户经理每年评聘申报时间是怎样的？

根据文件内容，客户经理每年评聘申报时间如下：

第十一条 每年一月份为客户经理评聘的申报时间，由分行人力资源部、个人业务部每年二月份组织统一的资格考试。考试合格者由分行颁发个金客户经理资格证书，其有效期为一年。

因此，客户经理每年评聘的申报时间是一月份。

CASE: DeepSeek + Faiss搭建本地知识库检索

Thinking: 项目的架构与技术选型是什么？

1. RAG架构

检索（Retrieval）：使用向量相似度搜索从PDF文档中检索相关内容

增强（Augmentation）：将检索到的文档片段作为上下文

生成（Generation）：基于上下文和用户问题生成答案

2. 技术栈选择

- 向量数据库：Faiss作为 高效的向量检索
- 嵌入模型：阿里云DashScope的text-embedding-v1
- 大语言模型：deepseek-v3
- 文档处理：PyPDF2用于PDF文本提取

CASE: DeepSeek + Faiss搭建本地知识库检索

Thinking: 程序的逻辑结构是什么？

Step1: 文档预处理

PDF文件 → 文本提取 → 文本分割 → 页码映射

1) PDF文本提取

- 逐页提取文本内容
- 记录每行文本对应的页码信息
- 处理空页和异常情况

2) 文本分割策略

- 使用递归字符分割器
- 分割参数: `chunk_size=1000, chunk_overlap=200`
- 分割符优先级: 段落 → 句子 → 空格 → 字符

3) 页码映射处理

- 基于字符位置计算每个文本块的页码
- 使用众数统计确定文本块的主要来源页码
- 建立文本块与页码的映射关系

CASE: DeepSeek + Faiss搭建本地知识库检索

Step2: 知识库构建

文本块 → 嵌入向量 → Faiss索引 → 本地持久化

1) 向量数据库构建

- 使用DashScope嵌入模型生成向量
- 将向量存储到Faiss索引结构

2) 数据持久化

- 保存Faiss索引文件（.faiss）
- 保存元数据信息（.pkl）
- 保存页码映射关系（page_info.pkl）

Step3: 问答查询

用户问题 → 向量检索 → 文档组合 → LLM生成 → 答案输出

1) 相似度检索

- 将用户问题转换为向量
- 在Faiss中搜索最相似的文档块，返回Top-K相关文档

2) 问答链处理

- 使用LangChain的load_qa_chain
- 采用 stuff 策略组合文档
- 将组合后的上下文和问题发送给LLM

3) 答案生成与展示

CASE: DeepSeek + Faiss搭建本地知识库检索

```
from PyPDF2 import PdfReader
from langchain.chains.question_answering import load_qa_chain
from langchain_community.callbacks.manager import get_openai_callback
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import DashScopeEmbeddings
from langchain_community.vectorstores import FAISS
from typing import List, Tuple
```

```
def extract_text_with_page_numbers(pdf) -> Tuple[str, List[int]]:
```

```
    """
```

从PDF中提取文本并记录每行文本对应的页码

参数:

pdf: PDF文件对象

返回:

text: 提取的文本内容

page_numbers: 每行文本对应的页码列表

```
    """
```

```
    text = ""
```

```
    page_numbers = []
```

```
    for page_number, page in enumerate(pdf.pages, start=1):
```

```
        extracted_text = page.extract_text()
```

```
        if extracted_text:
```

```
            text += extracted_text
```

```
            page_numbers.extend([page_number]
```

```
*)
```

```
len(extracted_text.split("\n")))
```

```
        else:
```

```
            Logger.warning(f"No text found on page {page_number}.")
```

```
    return text, page_numbers
```


CASE: DeepSeek + Faiss搭建本地知识库检索

```
def process_text_with_splitter(text: str, page_numbers: List[int]) -> FAISS:
```

```
    """
```

处理文本并创建向量存储

参数:

text: 提取的文本内容

page_numbers: 每行文本对应的页码列表

返回:

knowledgeBase: 基于FAISS的向量存储对象

```
    """
```

创建文本分割器, 用于将长文本分割成小块

```
text_splitter = RecursiveCharacterTextSplitter(
```

```
    separators=["\n\n", "\n", ".", " ", ""],
```

```
    chunk_size=1000,
```

```
    chunk_overlap=200,
```

```
    length_function=len,
```

```
)
```

```
# 分割文本
```

```
chunks = text_splitter.split_text(text)
```

```
print(f"文本被分割成 {len(chunks)} 个块。")
```

```
# 创建嵌入模型
```

```
embeddings = DashScopeEmbeddings(
```

```
    model="text-embedding-v1",
```

```
    dashscope_api_key=DASHSCOPE_API_KEY,
```

```
)
```

```
# 从文本块创建知识库
```

```
knowledgeBase = FAISS.from_texts(chunks, embeddings)
```

```
print("已从文本块创建知识库。")
```

```
# 存储每个文本块对应的页码信息
```

```
knowledgeBase.page_info = {chunk: page_numbers[i] for i,
chunk in enumerate(chunks)}
```

```
return knowledgeBase
```

CASE: DeepSeek + Faiss搭建本地知识库检索

```
# 读取PDF文件
pdf_reader = PdfReader('./浦发上海浦东发展银行西安分行个金客户经理考核
办法.pdf')
# 提取文本和页码信息
text, page_numbers = extract_text_with_page_numbers(pdf_reader)
text
```

'百度文库 - 好好学习，天天向上 \n-1 上海浦东发展银行西安分行 \n个金客户经理管理考核暂行办法 \n \n \n第一章 总 则 \n第一条 为保证我分行个金客户经理制的顺利实施，有效调动个\n金客户经理的积极性，促进个金业务快速、稳定地发展，根据总行《上\n海浦东发展银行个人金融营销体系建设方案（试行）》要求，特制定\n《上海浦东发展银行西安分行个金客户经理管理考核暂行办法（试\n行）》（以下简称本办法）。 \n第二条 个金客户经理系指各支行（营业部）从事个人金融产品\n营销与市场开拓，为我行个人客户提供综合银行服务的我行市场人\n员。 \n第三条 考核内容分为两大类，即个人业绩考核、工作质量考核。 \n个人业绩包括个人资产业务、负债业务、卡业务。工作质量指个人业\n务的资产质量。 \n第四条 为规范激励规则，客户经理的技术职务和薪资实行每年\n考核浮动。客户经理的奖金实行每季度考核浮动，即客户经理按其考\n核内容得分与行员等级结合，享受对应的行员等级待遇。 \n 百度文库 - 好好学习，天天向上 \n-2 第二章 职位设置与职责 \n第五条 个金客户经理职位设置为：客户经理助理、客户经理、\n高级客户经理、资深客户经理。 \n第六条 个金客户经理的基本职责： \n（一） 客户开发。研究客户信息、联系与选择客户、与客户建.....

CASE: DeepSeek + Faiss搭建本地知识库检索

```
print(f"提取的文本长度: {len(text)} 个字符。")
```

```
# 处理文本并创建知识库
```

```
knowledgeBase = process_text_with_splitter(text, page_numbers)
```

```
knowledgeBase
```

提取的文本长度: 3881 个字符。

文本被分割成 5 个块。

已从文本块创建知识库。

```
<langchain_community.vectorstores.faiss.FAISS at 0x170ab59f7d0>
```

CASE: DeepSeek + Faiss搭建本地知识库检索

```
from langchain_community.llms import Tongyi
llm = Tongyi(model_name="deepseek-v3",
dashscope_api_key=DASHSCOPE_API_KEY)
# 设置查询问题
query = "客户经理被投诉了，投诉一次扣多少分"
query = "客户经理每年评聘申报时间是怎样的？"
if query:
    # 执行相似度搜索，找到与查询相关的文档
    docs = knowledgeBase.similarity_search(query)

# 加载问答链
chain = load_qa_chain(llm, chain_type="stuff")

# 准备输入数据
input_data = {"input_documents": docs, "question": query}
```

```
# 使用回调函数跟踪API调用成本
with get_openai_callback() as cost:
    # 执行问答链
    response = chain.invoke(input=input_data)
    print(f"查询已处理。成本: {cost}")
    print(response["output_text"])
    print("来源:")

# 记录唯一的页码
unique_pages = set()
# 显示每个文档块的来源页码
for doc in docs:
    text_content = getattr(doc, "page_content", "")
    source_page = knowledgeBase.page_info.get(
        text_content.strip(), "未知"
    )
    if source_page not in unique_pages:
        unique_pages.add(source_page)
        print(f"文本块页码: {source_page}")
```

Summary

1. PDF文本提取与处理

使用PyPDF2库的PdfReader从PDF文件中提取文本

在提取过程中记录每行文本对应的页码，便于后续溯源

使用RecursiveCharacterTextSplitter将长文本分割成小块，便于向量化处理

2. 向量数据库构建

使用DashScopeEmbeddings将文本块转换为向量表示

使用FAISS向量数据库存储文本向量，支持高效的相似度搜索

为每个文本块保存对应的页码信息，实现查询结果溯源

3. 语义搜索与问答链

基于用户查询，使用similarity_search在向量数据库中检索相关文本块

使用Qwen语言模型和load_qa_chain构建问答链

将检索到的文档和用户问题作为输入，生成回答

4. 成本跟踪与结果展示

如果是openai模型，可以用get_openai_callback跟踪API调用成本

展示问答结果和来源页码，方便用户验证信息

打卡：创建你的RAG问答 (LangChain)



结合你的业务场景，创建你的RAG问答 (LangChain+DeepSeek+Faiss)

Step1, 收集整理知识库 (客户经理考核办法.pdf只是示例，用你的PDF进行检索)

Step2, 从PDF中提取文本并记录每行文本对应的页码

Step3, 处理文本并创建向量存储

Step4, 执行相似度搜索，找到与查询相关的文档

Step5, 使用问到链对用户问题进行回答 (使用你的DASHSCOPE_API_KEY)

Step6, 显示每个文档块的来源页码 (当前页码来源有问题，可以用Cursor完善)

CASE: DeepSeek + Faiss搭建本地知识库检索

Thinking: 如果得到的文本块的页码不对, 如何调整?

这个文本块页码不对, 帮我查看 文本块页码的计算逻辑, 是否有问题

Thought for 5 seconds

Searched codebase for "文本块页码的计算逻辑"

Read chatpdf-faiss.py

问题找到了。当前页码计算逻辑有以下问题:

1. 在 `extract_text_with_page_numbers` 函数中, 页码是按行文本进行记录的:

python

▷ Apply to chatpdf-fais...

```
page_numbers.extend([page_number] * len(extracted_text.split("\n")))
```

这里对每个PDF页面提取的文本按换行符分割, 然后为每一行都分配当前页码。

2. 但在 `process_text_with_splitter` 函数中, 文本被分成块时使用:

python

▷ Apply to chatpdf-fais...

```
page_info = {chunk: page_numbers[i] for i, chunk in enumerate(chunks)}
```

这段代码假设分割后的chunks数量与原始按行记录的page_numbers长度相同, 直接使用索引*i*来获取页码。

但实际上, `RecursiveCharacterTextSplitter` 分割器会按照多种分隔符 (`\n\n`, `\n`, `.`, `,`, ````) 将文本分割成chunk_size大小的块, 与原始按行分割的对应关系已经不存在了。

解决方案是修改页码记录逻辑, 使页码能够正确对应到分割后的文本块:

解决方案是修改页码记录逻辑, 使页码能够正确对应到分割后的文本块:

chatpdf-faiss.py +32 -1 ✓

```
# 修正页码逻辑: 为每个文本块找到最频繁出现的页码
page_info = {}
for chunk in chunks:
    # 找到文本块在原始文本中的起始位置
    chunk_start = text.find(chunk)
    if chunk_start == -1:
        # 如果找不到精确匹配, 可能是因为文本分割的边界问题
        # 使用一个默认页码 (例如第一页) 或标记为未知
        page_info[chunk] = "未知"
        continue

    # 计算文本块结束位置
    chunk_end = chunk_start + len(chunk)
```

LangChain中的问答链

LangChain问答链中的4种chain_type:

1) stuff

直接把文档作为prompt输入给OpenAI

2) map_reduce

对于每个chunk做一个prompt（回答或者摘要），然后再做合并

3) refine

在第一个chunk上做prompt得到结果，然后合并下一个文件再输出结果

4) map_rerank

对每个chunk做prompt，然后打个分，然后根据分数返回最好的文档中的结果

```
query = "客户经理每年评聘申报时间是怎样的？"
```

```
if query:
```

```
    # 执行相似度搜索，找到与查询相关的文档
```

```
    docs = knowledgeBase.similarity_search(query,k=10)
```

```
    # 加载问答链
```

```
    chain = load_qa_chain(llm, chain_type="stuff")
```

```
    # 准备输入数据
```

```
    input_data = {"input_documents": docs, "question": query}
```

```
    # 使用回调函数跟踪API调用成本
```

```
    with get_openai_callback() as cost:
```

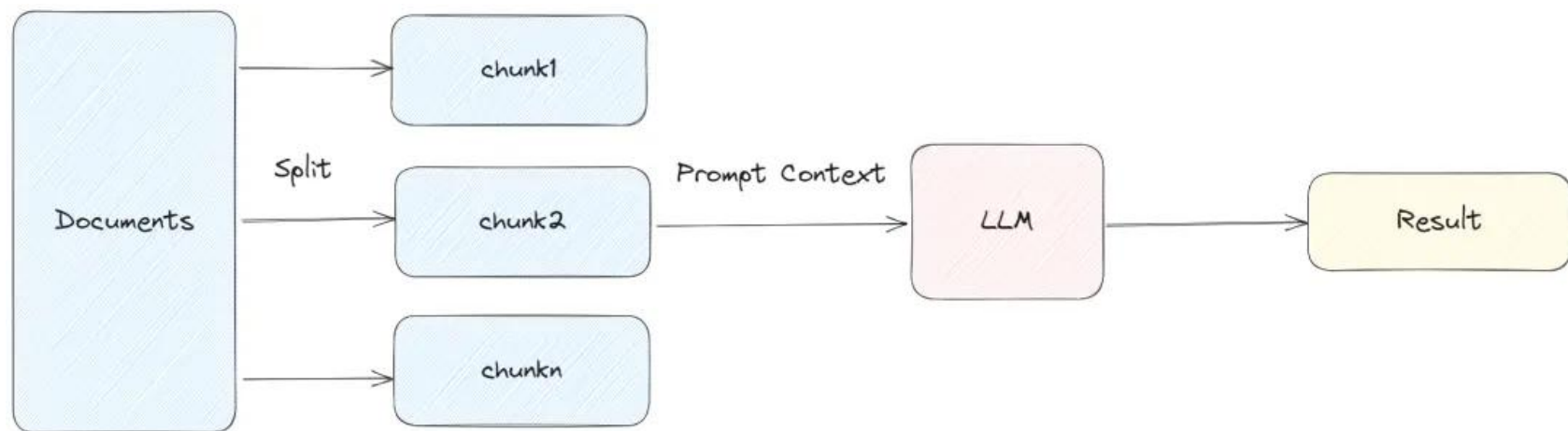
```
        # 执行问答链
```

```
        response = chain.invoke(input=input_data)
```

```
        print(f"查询已处理。成本: {cost}")
```

```
        print(response["output_text"])
```

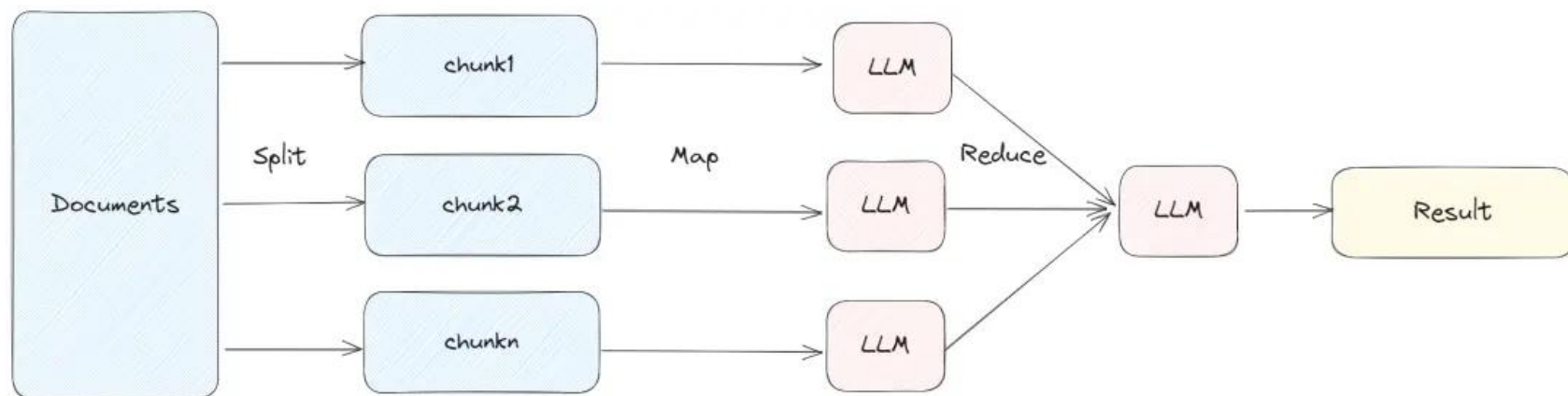

LangChain中的问答链



1) stuff

适合文档拆分的比较小，一次获取文档比较少的情况

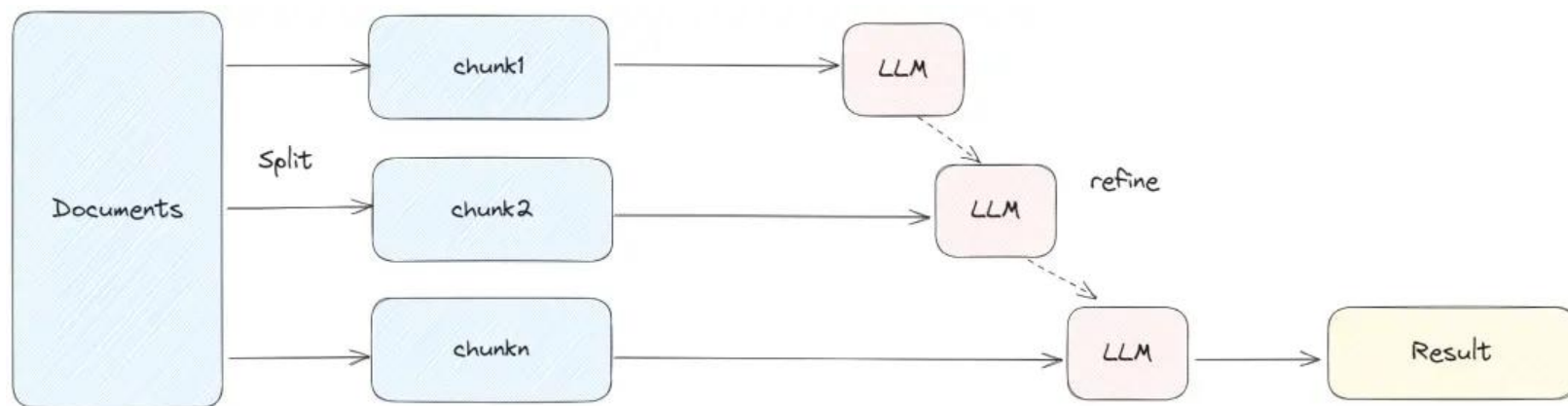
调用 LLM 的次数也比较少，能使用 stuff 的就使用这种方式。



2) map_reduce

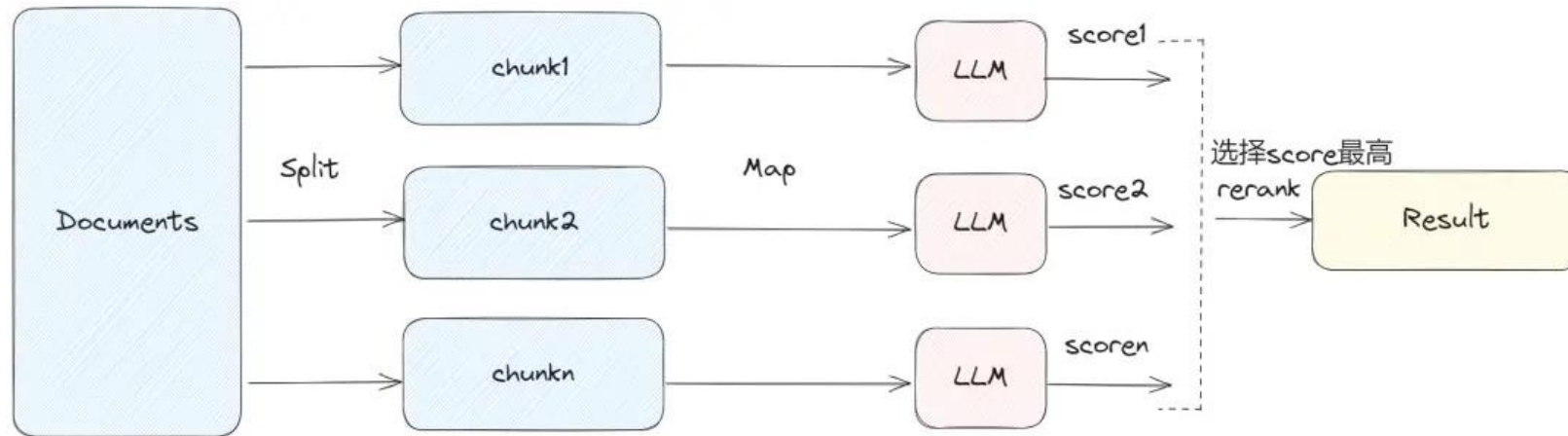
将每个 document 单独处理，可以并发进行调用。但是每个文档之间缺少上下文。

LangChain中的问答链



3) refine

Refine 这种方式能部分保留上下文，以及 token 的使用能控制在一定范围。



4) map_rerank

会大量地调用 LLM，每个 document 之间是独立处理

Q&A

Thinking: 如果LLM可以处理无限上下文了，RAG还有意义吗？

效率与成本: LLM处理长上下文时计算资源消耗大，响应时间增加。RAG通过检索相关片段，减少输入长度。

知识更新: LLM的知识截止于训练数据，无法实时更新。RAG可以连接外部知识库，增强时效性。

可解释性: RAG的检索过程透明，用户可查看来源，增强信任。LLM的生成过程则较难追溯。

定制化: RAG可针对特定领域定制检索系统，提供更精准的结果，而LLM的通用性可能无法满足特定需求。

数据隐私: RAG允许在本地或私有数据源上检索，避免敏感数据上传云端，适合隐私要求高的场景。

=> 结合LLM的生成能力和RAG的检索能力，可以提升整体性能，提供更全面、准确的回答。

Query改写

Query改写

Thinking: 为什么需要 Query 改写?

RAG 的核心在于“检索-生成”。如果第一步“检索”就走偏了，那么后续的“生成”质量也会降低。

用户提出的问题往往是口语化的、承接上下文的、模糊的，甚至是包含了情绪的。

而知识库里的文本（切片/Chunks）通常是陈述性的、客观的。

=> 需要一个翻译官的角色，将用户的“口语化查询”转换成“书面化、精确的检索语句”

Thinking: 如何针对不同类型的Query进行改写?

通过精心设计的 Prompt 来引导 LLM完成这项任务。

Query改写（上下文依赖型）

```
"""上下文依赖型Query改写"""
```

```
instruction = """
```

你是一个智能的查询优化助手。请分析用户的当前问题以及前序对话历史，判断当前问题是否依赖于上下文。

如果依赖，请将当前问题改写成一个独立的、包含所有必要上下文信息的完整问题。

如果不依赖，直接返回原问题。

```
"""
```

```
prompt = f"""
```

```
### 指令 ###
```

```
{instruction}
```

```
### 对话历史 ###
```

```
{conversation_history}
```

```
### 当前问题 ###
```

```
{current_query}
```

```
### 改写后的问题 ###
```

```
"""
```

对话历史:

用户: "我想了解一下上海迪士尼乐园的最新项目。"

AI: "上海迪士尼乐园最新推出了'疯狂动物城'主题园区，这里有朱迪警官和尼克狐的互动体验。"

用户: "这个园区有什么游乐设施？"

AI: "'疯狂动物城'园区目前有疯狂动物城警察局、朱迪警官训练营和尼克狐的冰淇淋店等设施。"

当前查询: 还有其他设施吗？

改写结果: 除了疯狂动物城警察局、朱迪警官训练营和尼克狐的冰淇淋店之外，'疯狂动物城'园区还有其他设施吗？

Query改写 (对比型)

```
"""对比型Query改写"""
```

```
instruction = """
```

你是一个查询分析专家。请分析用户的输入和相关的对话上下文，识别出问题中需要进行比较的多个对象。

然后，将原始问题改写成一个更明确、更适合在知识库中检索的对比性查询。

```
"""
```

```
prompt = f"""
```

```
### 指令 ###
```

```
{instruction}
```

```
### 对话历史/上下文信息 ###
```

```
{context_info}
```

```
### 原始问题 ###
```

```
{query}
```

```
### 改写后的查询 ###
```

```
"""
```

对话历史:

用户: "我想了解一下上海迪士尼乐园的最新项目。"

AI: "上海迪士尼乐园最新推出了疯狂动物城主题园区，还有蜘蛛侠主题园区"

当前查询: 哪个游玩的时间比较长，比较有趣

改写结果: 哪个游玩时间更长、更有趣: 上海迪士尼乐园的疯狂动物城主题园区和蜘蛛侠主题园区?

Query改写 (模糊指代型)

```
"""模糊指代型Query改写"""
```

```
instruction = """
```

你是一个消除语言歧义的专家。请分析用户的当前问题和对话历史，找出问题中 "都"、"它"、"这个" 等模糊指代词具体指向的对象。

然后，将这些指代词替换为明确的对象名称，生成一个清晰、无歧义的新问题。

```
"""
```

```
prompt = f"""
```

```
### 指令 ###
```

```
{instruction}
```

```
### 对话历史 ###
```

```
{conversation_history}
```

```
### 当前问题 ###
```

```
{current_query}
```

```
### 改写后的问题 ###
```

```
"""
```

对话历史:

用户: "我想了解一下上海迪士尼乐园和香港迪士尼乐园的烟花表演。"

AI: "好的，上海迪士尼乐园和香港迪士尼乐园都有精彩的烟花表演。"

当前查询: 都什么时候开始?

改写结果: 上海迪士尼乐园和香港迪士尼乐园的烟花表演都什么时候开始?

Query改写 (多意图型)

```
"""多意图型Query改写 - 分解查询"""
```

```
instruction = """
```

你是一个任务分解机器人。请将用户的复杂问题分解成多个独立的、可以单独回答的简单问题。以JSON数组格式输出。

```
"""
```

```
prompt = f"""
```

```
### 指令 ###
```

```
{instruction}
```

```
### 原始问题 ###
```

```
{query}
```

```
### 分解后的问题列表 ###
```

请以JSON数组格式输出，例如：["问题1", "问题2", "问题3"]

```
"""
```

原始查询: 门票多少钱? 需要提前预约吗? 停车费怎么收?

分解结果: ['门票多少钱?', '需要提前预约吗?', '停车费怎么收?']

Query改写 (反问型)

```
"""反问型Query改写"""
```

```
instruction = """
```

你是一个沟通理解大师。请分析用户的反问或带有情绪的陈述，识别其背后真实的意图和问题。

然后，将这个反问改写成一个中立、客观、可以直接用于知识库检索的问题。

```
"""
```

```
prompt = f"""
```

```
### 指令 ###
```

```
{instruction}
```

```
### 对话历史 ###
```

```
{conversation_history}
```

```
### 当前问题 ###
```

```
{current_query}
```

```
### 改写后的问题 ###
```

```
"""
```

对话历史:

用户: "你好, 我想预订下周六上海迪士尼乐园的门票。"

AI: "正在为您查询... 查询到下周六的门票已经售罄。"

用户: "售罄是什么意思? 我朋友上周去还能买到当天的票。"

当前查询: 这不会也要提前一个月预订吧?

改写结果: 迪士尼乐园门票是否需要提前一个月预订?

Query改写 (意图识别)

"""自动识别Query类型并进行改写"""

instruction = """

你是一个智能的查询分析专家。请分析用户的查询，识别其属于以下哪种类型：

1. 上下文依赖型 - 包含"还有"、"其他"等需要上下文理解的词汇
2. 对比型 - 包含"哪个"、"比较"、"更"、"哪个更好"、"哪个更"等比较词汇
3. 模糊指代型 - 包含"它"、"他们"、"都"、"这个"等指代词
4. 多意图型 - 包含多个独立问题，用"、"或"? "分隔
5. 反问型 - 包含"不会"、"难道"等反问语气

说明：如果同时存在多意图型、模糊指代型，优先级为多意图型>模糊指代型

请返回JSON格式：

```
{  
    "query_type": "查询类型",
```

```
    "rewritten_query": "改写后的查询",
```

```
    "confidence": "置信度(0-1)"
```

```
}
```

```
"""
```

```
    prompt = f"""
```

```
### 指令 ###
```

```
{instruction}
```

```
### 对话历史 ###
```

```
{conversation_history}
```

```
### 上下文信息 ###
```

```
{context_info}
```

```
### 原始查询 ###
```

```
{query}
```

```
### 分析结果 ###
```

```
"""
```

Query改写 (意图识别)

查询 1: 还有其他游乐项目吗?

识别类型: 上下文依赖型

改写结果: 除了之前提到的游乐项目之外, 还有哪些其他游乐项目?

置信度: 0.95

查询 2: 哪个园区更好玩?

识别类型: 模糊指代型

改写结果: 哪个园区更好玩? (请明确是哪些园区)

置信度: 0.95

查询 3: 都适合小朋友吗?

识别类型: 模糊指代型

改写结果: 哪些产品或活动适合小朋友?

置信度: 0.95

查询 4: 有什么餐厅? 价格怎么样?

识别类型: 多意图型

改写结果: 有哪些餐厅? 这些餐厅的价格怎么样?

置信度: 0.95

查询 5: 这不会也要排队两小时吧?

识别类型: 反问型

改写结果: 这需要排队两小时吗?

置信度: 0.95

打卡：Query改写



RAG 的核心在于“检索-生成”。如何通过Query改写让你的RAG检索更强大

- 上下文依赖型
- 对比型
- 模糊指代型
- 多意图型
- 反问型

你也可以通过意图识别，让AI自动分析是哪种类型，并进行改写

快来对用户的Query进行改写吧！

Query+联网搜索

Query+联网搜索

Thinking: 以迪士尼RAG助手为例，用户Query需要联网的情况都有哪些？

类型	关键词特征	示例查询	原因说明
时效性	最新、今天、现在、实时、当前	上海迪士尼乐园今天开放吗？	需获取当前时间的最新信息
价格信息	多少钱、价格、费用、票价	下周六的门票多少钱？	价格信息经常变动，需实时查询
营业信息	营业时间、开放时间、闭园时间、是否开放	迪士尼乐园现在开门吗？	营业状态可能因特殊情况调整
活动信息	活动、表演、演出、节日、庆典	最近有什么特别活动？	活动信息具有时效性和动态性
天气信息	天气、下雨、温度	明天去迪士尼天气怎么样？	天气信息需要实时获取
交通信息	怎么去、交通、地铁、公交	从浦东机场怎么去迪士尼？	交通信息可能因施工、活动等变化
预订信息	预订、预约、购票、订票	需要提前多久预订？	预订政策可能随时调整
实时状态	排队、拥挤、人流量	现在人多不多？	实时状态需即时获取

Query+联网搜索（识别）

核心功能1：识别查询是否需要联网搜索

```
def identify_web_search_needs(query, conversation_history):
```

输入：

- query: 用户查询
- conversation_history: 对话历史上下文

输出：

- need_web_search: 是否需要联网搜索
- search_reason: 搜索原因
- confidence: 置信度(0-1)

Query+联网搜索 (识别)

```
instruction = ""
```

你是一个智能的查询分析专家。请分析用户的查询，判断是否需要联网搜索来获取最新、最准确的信息。

需要联网搜索的情况包括：

1. 时效性信息 - 包含"最新"、"今天"、"现在"、"实时"、"当前"等时间相关词汇
2. 价格信息 - 包含"多少钱"、"价格"、"费用"、"票价"等价格相关词汇
3. 营业信息 - 包含"营业时间"、"开放时间"、"闭园时间"、"是否开放"等营业状态
4. 活动信息 - 包含"活动"、"表演"、"演出"、"节日"、"庆典"等动态信息
5. 天气信息 - 包含"天气"、"下雨"、"温度"等天气相关
6. 交通信息 - 包含"怎么去"、"交通"、"地铁"、"公交"等交通方式
7. 预订信息 - 包含"预订"、"预约"、"购票"、"订票"等预订相关
8. 实时状态 - 包含"排队"、"拥挤"、"人流量"等实时状态

Query+联网搜索 (识别)

请返回JSON格式:

```
{  
  "need_web_search": true/false,  
  "search_reason": "需要搜索的原因",  
  "confidence": "置信度(0-1)"  
}  
""
```

```
prompt = f"""
```

指令

```
{instruction}
```

对话历史

```
{conversation_history}
```

用户查询

```
{query}
```

分析结果

```
""
```

Query+联网搜索（改写）

核心功能2：为联网搜索改写查询

```
def rewrite_for_web_search(query, search_type="general"):
```

输入：

- query: 原始查询
- search_type: 搜索类型

输出：

- rewritten_query: 改写后的查询
- search_keywords: 搜索关键词列表
- search_intent: 搜索意图
- suggested_sources: 建议搜索来源

Query+联网搜索（改写）

```
instruction = ""
```

你是一个专业的搜索查询优化专家。请将用户的查询改写为更适合搜索引擎检索的形式。

改写技巧：

1. 添加具体地点 - 如"上海迪士尼乐园"、"香港迪士尼乐园"
2. 添加时间范围 - 如"2024年"、"今天"、"本周"
3. 使用关键词组合 - 将长句拆分为关键词
4. 添加搜索意图 - 明确搜索目的
5. 去除口语化表达 - 转换为标准搜索词
6. 添加相关词汇 - 增加同义词或相关词

Query+联网搜索（改写）

请返回JSON格式：

```
{  
  "rewritten_query": "改写后的搜索查询",  
  "search_keywords": ["关键词1", "关键词2", "关键词3"],  
  "search_intent": "搜索意图",  
  "suggested_sources": ["建议搜索的网站类型"]  
}
```

"""

```
prompt = f"""
```

指令

```
{instruction}
```

原始查询

```
{query}
```

搜索类型

```
{search_type}
```

改写结果

"""

Query+联网搜索（生成搜索策略）

核心功能3：生成搜索策略

`def generate_search_strategy(query, search_type="general"):`

输入：

- query: 用户查询
- search_type: 搜索类型

输出：

- primary_keywords: 主要关键词
- extended_keywords: 扩展关键词
- search_platforms: 搜索平台
- search_tips: 搜索技巧
- verification_methods: 验证方法

Query+联网搜索（生成搜索策略）

```
instruction = f"""
```

你是一个搜索策略专家。请为用户的查询制定详细的搜索策略。

当前日期: {current_date}

搜索策略包括:

1. 主要搜索词 - 核心关键词
2. 扩展搜索词 - 相关词汇和同义词
3. 搜索网站 - 推荐的搜索平台
4. 时间范围 - 具体的搜索时间范围

Query+联网搜索（生成搜索策略）

请返回JSON格式：

```
{  
  "primary_keywords": ["主要关键词"],  
  "extended_keywords": ["扩展关键词"],  
  "search_platforms": ["搜索平台"],  
  "time_range": "具体的时间范围"  
}  
"""  
  
prompt = f"""  
### 指令 ###  
{instruction}
```

用户查询

```
{query}
```

搜索类型

```
{search_type}
```

搜索策略

```
"""
```


Query+联网搜索

对话历史:

用户: "我想去上海迪士尼乐园玩"

AI: "上海迪士尼乐园是一个很棒的选择！"

当前查询: 上海迪士尼乐园今天开放吗？现在人多不多？

✓ 需要联网搜索

搜索原因: 查询上海迪士尼乐园今天是否开放属于营业信息，'现在人多不多'涉及实时状态（人流量），两者都需要联网获取最新、准确的信息。

置信度: 0.98

改写查询: 上海迪士尼乐园 2024年今天开放时间及人流情况

搜索关键词: ['上海迪士尼乐园', '开放时间', '2024年', '人流量', '游客数量']

搜索意图: 获取上海迪士尼乐园今日是否开放以及当前游客密度信息，用于出行规划

Query+联网搜索

建议来源: ['官方旅游网站', '携程/飞猪等旅游平台', '大众点评或美团用户评价', '本地生活类公众号']

搜索策略:

- 主要关键词: ['上海迪士尼乐园今天开放吗? 现在人多不多? ']
- 扩展关键词: []
- 搜索平台: ['百度', '谷歌']
- 时间范围: 最近一周

Query+联网搜索

当前查询: 下周六的门票多少钱? 需要提前多久预订?

✓ 需要联网搜索

搜索原因: 查询下周六的门票价格和预订时间, 涉及价格信息和预订信息, 需要联网获取最新、最准确的数据。

置信度: 0.98

改写查询: 下周六上海迪士尼乐园门票价格及预订时间要求

搜索关键词: ['下周六', '上海迪士尼乐园', '门票价格', '预订时间', '提前多久预订']

搜索意图: 获取特定日期的门票价格和预订政策信息

建议来源: ['官方网站', '旅游预订平台 (如携程、飞猪)', '景点官方社交媒体账号']

搜索策略:

- 主要关键词: ['下周六的门票多少钱? 需要提前多久预订? ']

- 扩展关键词: []

- 搜索平台: ['百度', '谷歌']

- 时间范围: 最近一周

Query+联网搜索

如果后续可以使用 Tavily MCP进行具体的联网搜索，可以引导LLM生成具体的参数

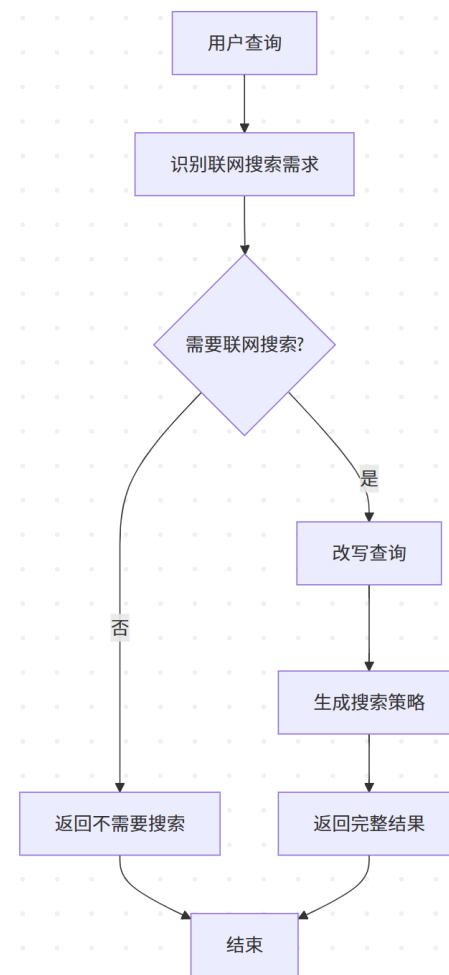
参数名	类型	是否必填	默认值	说明
query	string	✓	-	搜索关键词
search_depth	string	✗	"basic"	搜索深度，可选值："basic" 或 "advanced"
time_range	string	✗	"week"	时间范围，如 "day"、"week"、"month"、 "year" 或 "all"
max_results	int	✗	5	最大返回结果数量
include_images	bool	✗	FALSE	是否包含图片
include_answer	bool	✗	FALSE	是否包含由 LLM 生成的简短回答
include_raw_html	bool	✗	FALSE	是否包含原始 HTML 内容
topic	string	✗	"general"	搜索主题，可选值："general" 或 "news"
domains	list	✗	-	限定搜索的域名白名单，如 ["bbc.com", "techcrunch.com"]
exclude_domains	list	✗	-	排除的域名黑名单

打卡：Query+联网搜索



以迪士尼RAG助手为例，搭建Query+联网搜索的改写功能，方便后续进行RAG问答

- 设置联网搜索的使用场景，比如：时效性、天气等
- 识别逻辑：通过LLM，判断是否需要联网搜索
- 改写逻辑：通过LLM，对联网搜索的情况，进行查询改写
- 生成策略（可选）：可以提出更详细的平台，以及平台关键词，方便后续进行联网搜索





Thank You
Using data to solve problems